

RETO 02 - RA2 - UT3

Visualizaciones Barras y Pie

Ejercicio 1:

Dominando los Gráficos de Barras

Ejercicio 2:

Gráficos de Pie y Dona

Antonio Ferrer Martínez

Marzo 2026

04b__Tarea__Graficos__de__Barras

March 14, 2026

1 Tarea paso a paso: Dominando los Gráficos de Barras

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre la generación de gráficos de barras mediante **Pandas**, **Seaborn** y **Matplotlib** (basado en lo aprendido en `4_grafico_de_barras.ipynb`).

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución de este Notebook en segundos. **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES)** En la vida real analítica (Data Science, Big Data), las IAs son asistentes geniales, pero el que diseña, entiende la arquitectura, detecta errores sutiles y toma las decisiones de valor eres *tú*. Si no entiendes qué hace un agrupamiento (`groupby`), qué parámetros exactos recibe un `barplot` o cómo parametrizar los ejes X e Y, serás incapaz de resolver problemas de negocio reales y novedosos. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Python, busca tu fallo y ¡aprende de él!**

1.1 Paso 1: Obtener el Dataset de Kaggle

Vamos a trabajar con el dataset de **Top 50 Universities in the U.S. (Forbes 2026)**.

1. Regístrate o inicia sesión en Kaggle y entra a este enlace: [Top 50 Universities Forbes 2026](#)
2. Haz clic en *Download* (te descargará un archivo `.zip` que contiene un archivo CSV).
3. Extrae, renombra el archivo CSV principal a `universities.csv` y muévelo directamente a tu subcarpeta `data/` (dentro de `02_Visualizaciones`).

1.2 Paso 2: Importar librerías

Importa `pandas`, `seaborn` y `matplotlib.pyplot` en la siguiente celda como hemos visto en clase.

```
[1]: # Importar la librería pandas para manipulación y análisis de datos
import pandas as pd
# Importar seaborn para visualización estadística atractiva
import seaborn as sns
# Importar pyplot de matplotlib para crear gráficos básicos y personalizar_
    ↪ figuras
import matplotlib.pyplot as plt
```

1.3 Paso 3: Carga de Datos y Exploración General

Usa **pandas** para leer el archivo CSV (**universities.csv**) que has guardado en la carpeta **data/**. Guárdalo en la variable de DataFrame típica (**df**) y muestra las primeras 5 filas (**.head()**) para ver la composición de sus columnas (Name, State, Type, National Rank, Research Impact Score, etc).

```
[2]: # CARGA DE DATOS
# pd.read_csv() lee el archivo CSV (valores separados por comas) y lo convierte
# en un DataFrame (tabla de datos)
df = pd.read_csv("data/universities.csv") # Por defecto detecta las columnas
# por el encabezado del archivo
# df.head() muestra las primeras 5 filas del DataFrame para tener una vista
# rápida de los datos
df.head()
```

```
[2]:
```

	University_Name	National_Rank	Founded_Year	\
0	Massachusetts Institute of Technology (MIT)	1	1861	
1	Columbia University	2	1754	
2	Princeton University	3	1746	
3	Stanford University	4	1891	
4	University of California, Berkeley	5	1868	

	Institution_Type	State	Research_Impact_Score	Intl_Student_Ratio	\
0	Private	MA	100.0	91.6	
1	Private	NY	95.9	83.7	
2	Private	NJ	99.0	70.0	
3	Private	CA	99.5	73.5	
4	Public	CA	98.9	70.6	

	Employment_Rate
0	96.2
1	92.1
2	94.5
3	97.8
4	91.4

1.4 Paso 4: Gráfico de Barras Vertical con Pandas (.plot.bar)

Objetivo: Averiguar qué Estados (**State**) tienen mayor cantidad de universidades en el Top 50.

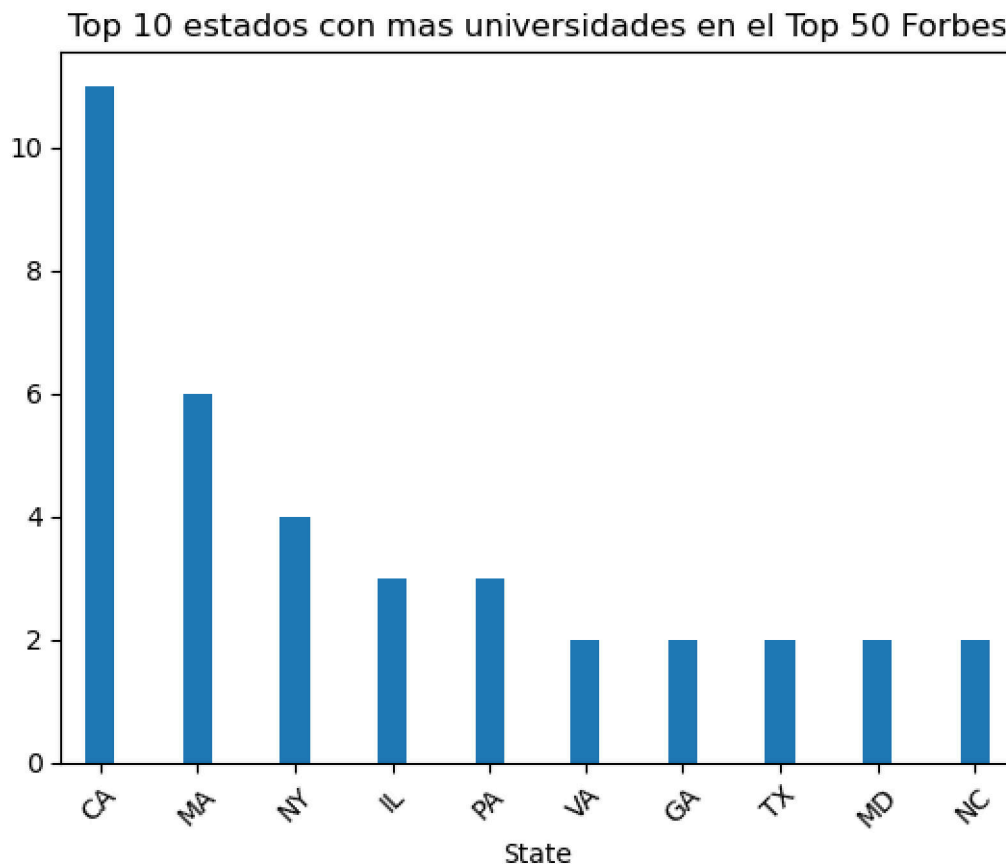
1. Agrupa el DataFrame usando la columna **State**.
2. Cuenta el número de universidades en cada estado (puedes usar **.size()** o contar una columna como **Name**).
3. Ordena los valores de mayor a menor y quédate con el top 10 de estados.
4. Sobre esta serie, llama a las librerías nativas de pandas para graficar un diagrama de barras vertical (**.plot.bar()**).
5. Configura un título y ajusta el parámetro **rot=0** o **rot=45** si los nombres del eje X aparecen

torcidos.

```
[3]: # Agrupamos por 'State' y aplicamos una función de agregación (.agg)
# específicamente, contamos el número de 'University_Name' para cada grupo
estados = df.groupby('State').agg({
    "University_Name" : "count"
})
# Renombramos la columna para que sea mas descriptiva
estados = estados.rename(columns={'University_Name': 'Cantidad'})
# Ordenamos de mayor a menor y nos quedamos con el top 10
estados = estados.sort_values('Cantidad', ascending=False).head(10)
# Mostramos el DataFrame resultante 'estados'
estados

# Gráfico de Barras con Pandas (por debajo usa matplotlib)
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.bar.html
# Llamamos al método .plot.bar() sobre nuestro DataFrame 'estados'
estados.plot.bar(
    # Columna a utilizar para el eje Y (altura de las barras)
    y='Cantidad',
    # Título del gráfico
    title='Top 10 estados con mas universidades en el Top 50 Forbes',
    # Ancho de las barras (0.3 es el 30% del espacio disponible para cada
    ↪ categoría)
    width=0.3,
    # rotación de las etiquetas del eje X (45 grados para que se lean bien)
    rot=45,
    # Argumento adicional: legend=False oculta la leyenda del gráfico ya que es
    ↪ redundante
    legend=False
)
```

```
[3]: <Axes: title={'center': 'Top 10 estados con mas universidades en el Top 50
Forbes'}, xlabel='State'>
```



1.5 Paso 5: Gráfico de Barras Horizontal con Seaborn (sns.barplot)

Objetivo: ¿Qué tipo de universidad (Type, público o privado) tiene de media (promedio) un mayor Research Impact Score (Impacto de Investigación)?

1. Llama a un gráfico de tipo barras en **Seaborn**. Recuerda que *Seaborn* (por defecto) ya te calcula el promedio cuando le pasas una variable categórica y otra numérica al graficar.
2. Haz el gráfico **horizontal**, es decir:
 - Eje X será el impacto de investigación (Research Impact Score).
 - Eje Y será el tipo de universidad (Type).
3. Usa una paleta de colores de tu elección (argumento `palette`).
4. Ajusta el tamaño de la figura (`plt.figure(figsize=...)`) si lo ves conveniente.
5. Ponle un título descriptivo.

```
[4]: # Gráfico de Barras con Seaborn
# https://seaborn.pydata.org/generated/seaborn.barplot.html
# sns.barplot() crea un gráfico de barras utilizando Seaborn
plt.figure(figsize=(10, 5))
sns.barplot(
    # Pasamos el DataFrame completo (Seaborn calcula la media automaticamente)
```

```

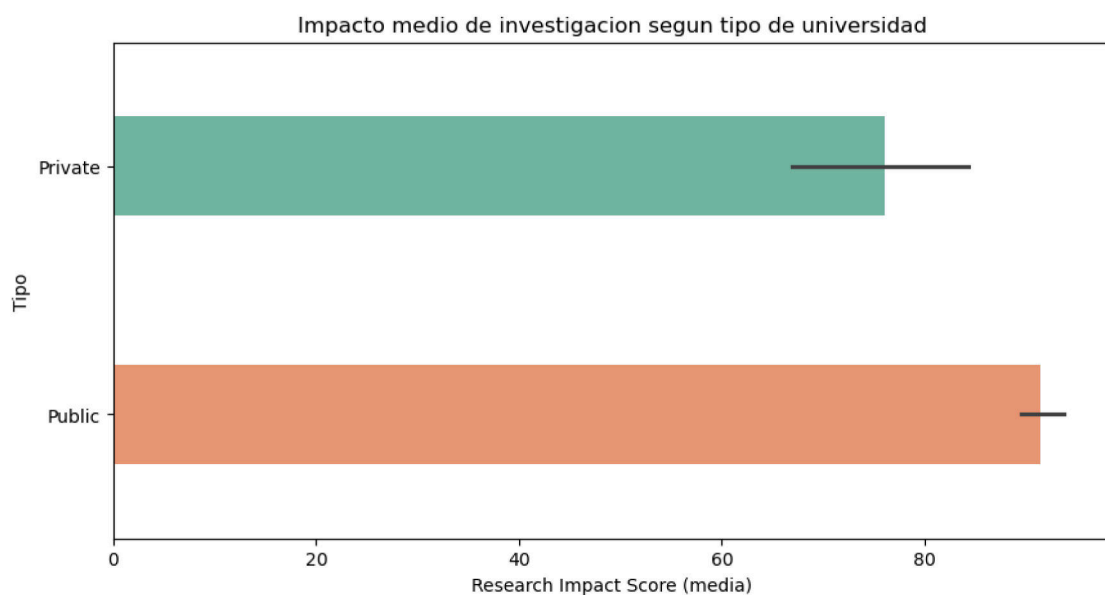
data=df,
# Indicamos qué columna irá en el eje X
x='Research_Impact_Score',
# Indicamos qué columna se usará para la altura en el eje Y
y='Institution_Type',
# 'hue' colorea las barras basándose en la columna especificada (aquí le
↳ damos un color distinto a cada barra)
hue='Institution_Type',
# Paleta de colores
palette='Set2',
# Grosor de las barras
width=0.4,
# legend=False oculta la leyenda de colores generada por el 'hue'
legend=False
# .set() permite configurar las etiquetas de los ejes y el título del gráfico
↳ de una sola vez
).set(title='Impacto medio de investigacion segun tipo de universidad',
↳ xlabel='Research Impact Score (media)', ylabel='Tipo')

```

```

[4]: [Text(0.5, 1.0, 'Impacto medio de investigacion segun tipo de universidad'),
Text(0.5, 0, 'Research Impact Score (media)'),
Text(0, 0.5, 'Tipo')]

```



1.6 Paso 6: Construcción manual de Gráfico de barras vertical con Matplotlib (plt.bar)

Objetivo: Crear tu propio gráfico usando Matplotlib mostrando el Top 5 de universidades con mayores tasas de empleo post-graduación (Post-Graduation Employment Rates).

(Ojo: Si esta columna viene como texto con '%', primero tendrás que limpiarla copiando y transformándola a numérico. Si es directamente un número, continúa)

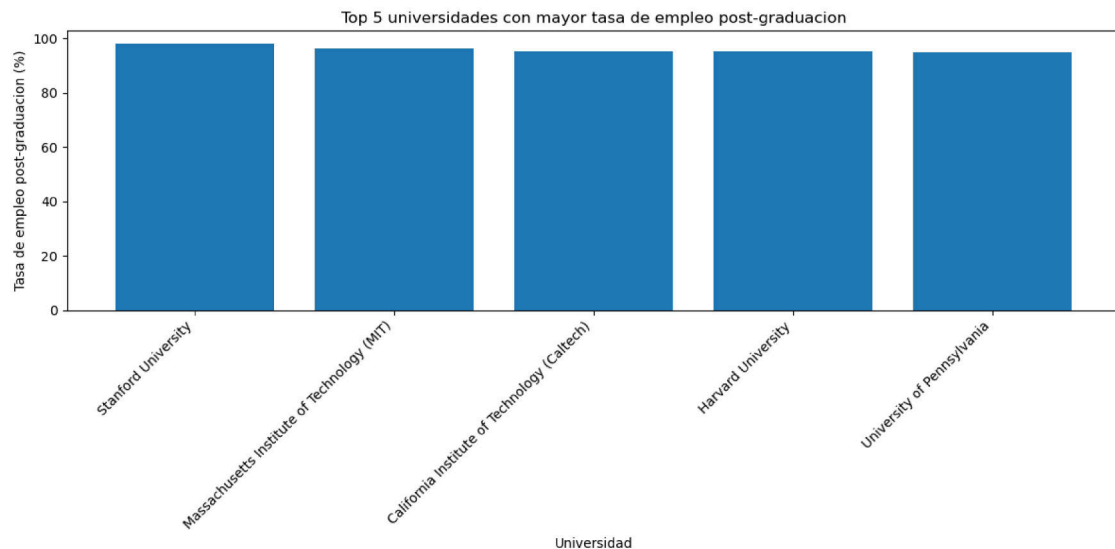
1. Ordena directamente el DataFrame completo de manera descendente según su Post-Graduation Employment Rates y selecciona el Top 5 (.head(5)).
2. Extrae los datos: Guarda los nombres de las Universidades (Name) y sus tasas de empleo en listas/variables separadas.
3. Configura plt.figure ampliando el tamaño horizontal para que quepan los nombres de las universidades.
4. Pinta barras simples introduciendo los dos ejes extraídos (plt.bar(x, y)).
5. Etiqueta títulos, Eje X, Eje Y explícitamente usando plt.xlabel(), plt.ylabel() y plt.title().
6. Para evitar que el texto de las universidades se superponga en el eje X, usa plt.xticks(rotation=45, ha="right").
7. Finaliza con plt.show()

```
[5]: # Gráfico de Barras verticales con Matplotlib sin usar pandas
# https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.bar.html

# Ordenamos el DataFrame por 'Employment_Rate' de mayor a menor y seleccionamos
# el Top 5
top5 = df.sort_values('Employment_Rate', ascending=False).head(5)

# plt.figure() inicializa una nueva figura.
# figsize=(12, 6) establece el tamaño de la figura en 12 pulgadas de ancho por
# 6 de alto.
plt.figure(figsize=(12, 6))
# plt.bar() crea un gráfico de barras verticales
plt.bar(
    # Categorías para el eje X
    top5['University_Name'],
    # Alturas correspondientes para el eje Y
    top5['Employment_Rate']
)
# plt.xlabel y plt.ylabel definen los textos de acompañamiento de los ejes
# correspondientes
plt.xlabel('Universidad')
plt.ylabel('Tasa de empleo post-graduacion (%)')
# Título del diagrama
plt.title('Top 5 universidades con mayor tasa de empleo post-graduacion')
# Rotamos las etiquetas del eje X para que no se superpongan
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
```

```
# plt.show() renderiza la ventana que muestra el gráfico finalmente  
plt.show()
```



1.7 Reflexión Final del Alumno

Tómate un rato. Como has podido notar en esta lección:

- En **pandas** llamamos un gráfico directamente al objeto como un método adherido a la tabla en memoria (`.plot.bar()`).
- En **Seaborn** construimos gráficos estadísticos pasándole el `DataFrame` y marcando los ejes como si le habláramos de forma humana.
- En **Matplotlib** pintamos capa sobre capa con pluma y papel como un lienzo crudo usando listas discretas de puntos X e Y.

Asegúrate que tu código de las celdas anteriores esté exhaustivamente comentado por ti en cada paso importante para justificar qué hace y que entiendes que el agrupamiento matemático inicial condiciona al propio dibujo.

1.7.1 Mi reflexión

Antes de hacer cualquier gráfico hay que agrupar los datos con `groupby` y contar, si no el gráfico no tiene qué representar. Con **pandas** es directo, le haces `.plot.bar()` al dataframe agrupado y listo. **Seaborn** es diferente porque él ya calcula la media sólo cuando le pasas una columna categórica y otra numérica, así que no hace falta agrupar antes. **Matplotlib** requiere más líneas de código pero a cambio controlas cada detalle del gráfico por separado. He usado `rot=45` en los `xticks` para que las etiquetas no se pisen entre ellas.

05b__Tarea__Grafico__Pie__Dona

March 14, 2026

1 Tarea paso a paso: Gráficos de Pie y Dona

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre la generación de gráficos de proporciones mediante **Pandas y Plotly Express** (basado en lo aprendido en `6_grafico_pie_dona.ipynb`).

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución de este Notebook en segundos. **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** En la vida real analítica (Data Science, Big Data), las IAs son asistentes geniales, pero el que diseña, entiende la arquitectura, detecta errores sutiles y toma las decisiones de valor eres *tú*. Si no entiendes cómo agrupar datos, qué espera un gráfico circular como entrada o cómo usar Plotly, serás incapaz de resolver problemas de negocio reales. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación, busca tu fallo y ¡aprende de él!**

1.1 Paso 1: Carga y Exploración de Datos

Vamos a trabajar con el dataset **IMDb Top 1000 Movies**.

1. Regístrate o inicia sesión en Kaggle y entra a este enlace: [IMDb Top 1000 Movies Dataset](#)
2. Haz clic en *Download* (te descargará un archivo `.zip` que contiene un archivo CSV).
3. Extrae, renombra el archivo CSV a `imdb_top_1000.csv` y muévelo directamente a tu subcarpeta `data/`.
4. Importa las librerías necesarias (`pandas`, `plotly.express`).
5. Lee el archivo CSV y guárdalo en un DataFrame llamado `df`.
6. Muestra las primeras 5 filas para recordar qué columnas tiene (Series_`Title`, `Certificate`, `Genre`, `IMDB_Rating`, etc).

```
[1]: import pandas as pd # Importa la libreria pandas para manipulacion y analisis
    ↪ de datos
import seaborn as sns # Importa seaborn para visualizacion de datos
import matplotlib.pyplot as plt # Importa matplotlib.pyplot para crear graficos
import plotly.express as px # Importa plotly.express para graficos interactivos

# CARGA DE DATOS
df = pd.read_csv("data/imdb_top_1000.csv") # Lee el archivo 'imdb_top_1000.csv'
    ↪ y lo guarda en el DataFrame 'df'. Por defecto detecta las columnas por el
    ↪ encabezado del archivo
```

```
df.head() # Muestra las primeras 5 filas del DataFrame 'df' para visualizar los
↳ datos
```

```
[1]:
```

	Poster_Link \
0	https://m.media-amazon.com/images/M/MV5BMDFkYT...
1	https://m.media-amazon.com/images/M/MV5BM2MyNj...
2	https://m.media-amazon.com/images/M/MV5BMTMxNT...
3	https://m.media-amazon.com/images/M/MV5BMWwMG...
4	https://m.media-amazon.com/images/M/MV5BMWU4N2...

	Series_Title	Released_Year	Certificate	Runtime \
0	The Shawshank Redemption	1994	A	142 min
1	The Godfather	1972	A	175 min
2	The Dark Knight	2008	UA	152 min
3	The Godfather: Part II	1974	A	202 min
4	12 Angry Men	1957	U	96 min

	Genre	IMDB_Rating \
0	Drama	9.3
1	Crime, Drama	9.2
2	Action, Crime, Drama	9.0
3	Crime, Drama	9.0
4	Crime, Drama	9.0

	Overview	Meta_score \
0	Two imprisoned men bond over a number of years...	80.0
1	An organized crime dynasty's aging patriarch t...	100.0
2	When the menace known as the Joker wreaks havo...	84.0
3	The early life and career of Vito Corleone in ...	90.0
4	A jury holdout attempts to prevent a miscarria...	96.0

	Director	Star1	Star2	Star3 \
0	Frank Darabont	Tim Robbins	Morgan Freeman	Bob Gunton
1	Francis Ford Coppola	Marlon Brando	Al Pacino	James Caan
2	Christopher Nolan	Christian Bale	Heath Ledger	Aaron Eckhart
3	Francis Ford Coppola	Al Pacino	Robert De Niro	Robert Duvall
4	Sidney Lumet	Henry Fonda	Lee J. Cobb	Martin Balsam

	Star4	No_of_Votes	Gross
0	William Sadler	2343110	28,341,469
1	Diane Keaton	1620367	134,966,411
2	Michael Caine	2303232	534,858,444
3	Diane Keaton	1129952	57,300,000
4	John Fiedler	689845	4,360,000

1.2 Paso 2: Gráfico de Pie Clásico con Pandas

Objetivo: Mostrar la proporción de películas según su clasificación por edades (Certificate).

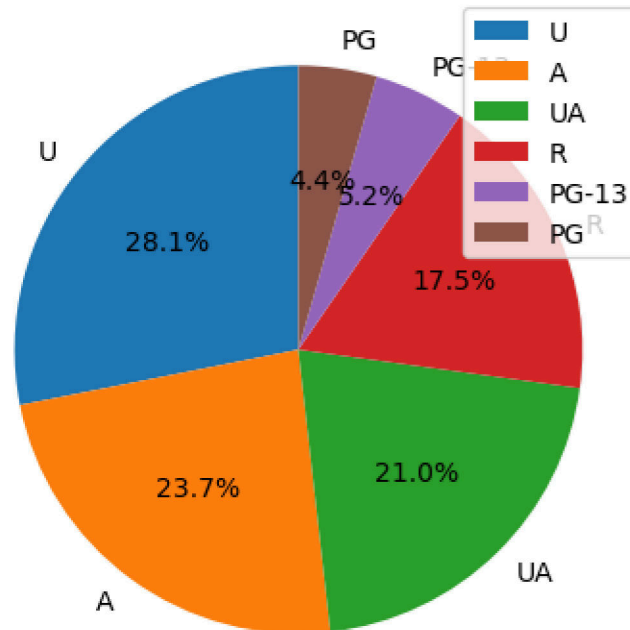
1. Utiliza `groupby` para agrupar el DataFrame por la columna `Certificate`.
2. Haz un conteo (usando por ejemplo `agg` sobre `Series_Title` o simplemente un `.count()`). Toma solo una columna resultante para que Pandas no intente graficar varias cosas.
3. (Opcional) Para que el gráfico no sea un caos visual con clasificaciones raras o vacías, filtra tu DataFrame agrupado quedándote solo con las clasificaciones con frecuencias más altas.
4. Usa el método `.plot.pie()` nativo de Pandas sobre los datos agrupados.
5. Incluye los porcentajes usando `autopct='%1.1f%%'` y un buen `startangle`.

```
[2]: cert = df.groupby('Certificate').agg({ # Crea un nuevo DataFrame 'cert'
    ↪agrupando por 'Certificate'
    "Series_Title" : "count" # Cuenta el número de películas para cada
    ↪clasificación
})
cert = cert.rename(columns={'Series_Title': 'Count'}) # Renombramos la columna
    ↪para que sea mas clara
# Nos quedamos solo con las 6 clasificaciones mas frecuentes para que el
    ↪grafico sea legible
cert = cert.sort_values('Count', ascending=False).head(6)
cert # Muestra el DataFrame 'cert' con el resultado

# Grafico de Pie con Pandas
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.pie.html
# https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.pie.
    ↪html#matplotlib.axes.Axes.pie
cert.plot.pie( # Crea un grafico de sectores (pie chart) usando la libreria
    ↪pandas
    y='Count', # Especifica que los valores para el grafico provienen de la
    ↪columna 'Count'
    startangle=90, # Rota el inicio del grafico 90 grados para que el primer
    ↪sector empiece arriba en lugar de a la derecha
    autopct='%1.1f%%' # Muestra automaticamente el calculo del porcentaje en
    ↪cada sector con 1 numero decimal
)
plt.title('Proporción de películas por clasificación (top 6)') # Establece el
    ↪titulo de la grafica
plt.ylabel('') # Quitamos la etiqueta del eje Y que en un pie no aporta nada
```

```
[2]: Text(0, 0.5, '')
```

Proporcion de peliculas por clasificacion (top 6)



1.3 Paso 3: Entrando en Interactividad: Gráfico de Pie con Plotly Express

Objetivo: Mostrar la proporción de películas agrupadas por su género (**Genre**).

(Nota: Como la columna **Genre** tiene varios géneros separados por comas, asume por simplicidad que toda la celda es la categoría, o si te atreves, crea una nueva columna **Main_Genre** quedándote solo con la primera palabra antes de la coma para hacer la limpieza)

1. Agrupa los datos por el género (**Genre** o tu nuevo **Main_Genre**) y cuenta las películas.
2. Quédate únicamente con el Top 10 de géneros más repetidos para que el gráfico sea legible.
3. **Importante:** Recuerda usar `.reset_index()` para que el índice de agrupamiento vuelva a ser una columna normal que Plotly pueda leer.
4. Llama a `px.pie(...)` pasándole el DataFrame agrupado, e indicando qué columna representa los nombres (**names**) y cuál representa los valores (**values**).
5. Añade un título descriptivo y usa `.show()` para renderizar el gráfico.

```
[3]: # La columna Genre tiene varios generos separados por coma ("Crime, Drama")
# Creamos una nueva columna Main_Genre quedandonos sólo con el primer genero
# ↪ antes de la coma
df['Main_Genre'] = df['Genre'].str.split(',').str[0].str.strip()

# Agrupamos por genero principal y contamos peliculas
generos = df.groupby('Main_Genre').agg({
```

```

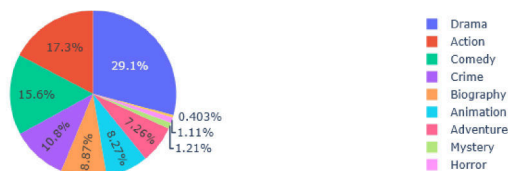
    "Series_Title" : "count" # Cuenta el numero de peliculas para cada genero
})
generos = generos.rename(columns={'Series_Title': 'Count'})
# Top 10 generos mas repetidos para que el pie sea legible
generos = generos.sort_values('Count', ascending=False).head(10)

# Para poder trabajar con Plotly, necesitamos poder pasarle el dataframe a sus
↳ métodos gráficos.
# Si queremos trabajar con los datos dentro de la variable 'generos', debemos
↳ antes desagrupar para que el índice 'Main_Genre' se vuelva una columna
↳ normal.
# reset_index() convierte el índice (que actualmente es 'Main_Genre') en una
↳ columna del DataFrame
generos = generos.reset_index()
# Mostramos el nuevo DataFrame 'generos'
generos

# Grafico de Pie o Tarta
# https://plotly.com/python/pie-charts/
fig = px.pie( # Genera un grafico circular utilizando la libreria Plotly
↳ Express llamando su metodo pie
    generos, # Le pasamos el DataFrame completo con todos los datos a pintar
    names='Main_Genre', # Utiliza la columna 'Main_Genre' para sacar las
↳ etiquetas (nombres de cada sector)
    values='Count', # Usa la columna 'Count' como valores para calcular las
↳ proporciones del perimetro
    title='Top 10 generos de peliculas - IMDb Top 1000' # Define el titulo
↳ principal que aparecera sobre el grafico
)
fig.show() # Expresa este grafico y lo muestra de forma interactiva en la
↳ pantalla

```

Top 10 generos de peliculas - IMDb Top 1000



1.4 Paso 4: Gráfico de Dona y Leyendas Personalizadas

Objetivo: Crear un gráfico de Dona mostrando el porcentaje del Top 5 de Directores con más películas en el ranking.

1. Agrupa los datos por la columna `Director` y cuenta sus registros.
2. Ordena los directores de mayor a menor y quédate solo con el Top 5.
3. Resetea el índice para que sea compatible con Plotly Express.
4. Genera el gráfico con `px.pie(...)` asegurándote de usar el parámetro `hole=0.5` para que se convierta en una Dona.
5. Cambia los textos de la leyenda o los *hovers* usando `labels={...}` para mayor legibilidad humana.
6. Opcionalmente, usa `fig.update_layout(showlegend=False)` para ocultar la leyenda o déjala visible si suma al contexto.

```
[4]: # Agrupamos por 'Director' y aplicamos una función de agregación (.agg)
directores = df.groupby('Director').agg({
    "Series_Title" : "count" # Cuenta el numero de peliculas para cada director
})
directores = directores.rename(columns={'Series_Title': 'Count'})
# Ordenamos de mayor a menor y nos quedamos con el Top 5
directores = directores.sort_values('Count', ascending=False).head(5)
# reset_index() convierte el índice (que actualmente es 'Director') en una
↳ columna del DataFrame
directores = directores.reset_index()
# Mostramos el DataFrame resultante
directores

# Grafico Donut o Dona con Plotly
# https://plotly.com/python/pie-charts/
fig = px.pie( # Llama a Plotly Express para crear un grafico circular
    directores, # Se pasan los datos base para plotear DataFrame
    names='Director', # Nombre de las secciones: establecemos las etiquetas
    ↳ usando la columna 'Director'
    values='Count', # Tamano de las secciones dependiendo de el valor numerico
    ↳ en esta columna
    title='Top 5 directores con mas peliculas en el ranking IMDb', # Configura
    ↳ el titulo descriptivo
    hole=0.5, # Crea un agujero vacio en el centro del 50% para que en lugar de
    ↳ un Pie sea un grafico de Dona
    labels={
        'Director': 'Director',
        'Count': 'Nº de peliculas' # Cambia la leyenda de los valores de 'Count'
    }
)
fig.show() # Presenta este grafico interactivo
```

Top 5 directores con mas peliculas en el ranking IMDb



1.5 Reflexión Final del Alumno

Tómate un rato. Como has visto:

- Los **gráficos de Pie/Dona** son excelentes para mostrar porcentajes o composiciones de un todo cuando las categorías son muy pocas.
- Requieren siempre una **agrupación y conteo previo** del conjunto de datos.
- Plotly introduce una capa interactiva muy superior a Pandas, siendo ideal para dashboards.

Asegúrate de que tu código esté adecuadamente comentado indicando qué hace cada agrupación y por qué configuras atributos visuales.

1.5.1 Mi reflexión

Para los gráficos de pie hay que agrupar y contar antes de dibujar nada. He filtrado a las categorías más frecuentes en cada caso porque si no el gráfico queda lleno de trozos pequeños y no se lee bien. Lo del `reset_index` es necesario para que Plotly pueda usar la columna de agrupación como names, ya que después del `groupby` esa columna pasa a ser el índice. Plotly es bastante mejor que el pie de pandas porque es interactivo, y para convertirlo en dona basta con añadir `hole=0.5`. He usado `labels` para que las leyendas sean más claras.